Authors:

Maria Zakaryan (zakaryan.2129276@studenti.uniroma1.it)

Louisa Korshunova (korshunova.2053311@studenti.uniroma1.it)

Program: Bioinformatics

Course: AI-LAB – Computer Vision, Signal Processing, and Natural Language Processing

Institution: Department of Computer Science, Sapienza University of Rome

DeepCough: COVID-19 Detection from Voice and Cough Recordings Using Convolutional Neural Networks

Abstract:

The COVID-19 pandemic highlighted the need for diagnostic tools that are fast, accessible, and easy to use. While methods such as PCR and antigen tests provide reliable results, they can be costly, time-consuming, and difficult to access during large-scale health emergencies. This has motivated the search for alternative screening methods that can be performed remotely and without invasive procedures. To that end, this project aims to distinguish between healthy individuals and COVID-19 positive patients using cough recordings. Using the COVID-19 Cough Sounds dataset from Kaggle, respiratory audio samples are transformed into Mel-spectrograms, allowing the problem to be approached as an image classification task. An adapted ResNet18 architecture is then used to analyze these spectrogram images and identify acoustic patterns associated with COVID-19. To improve performance on a relatively small and noisy dataset, data augmentation and normalization are employed. The developed system achieved a validation accuracy of 86.7%, demonstrating that computer vision methods applied to respiratory sounds can provide an effective and non-invasive solution for preliminary health screening.

1. Introduction

1.1 The Problem and Context

Major global health crises often expose the limitations of existing healthcare systems. When large numbers of people require testing at the same time, providing fast and widespread diagnosis becomes a significant challenge. Although PCR and rapid antigen tests are highly reliable and remain the standard methods for detecting COVID-19, they are not always easy to deploy on a large scale. Their use depends on specialized equipment, trained personnel, and the availability of medical supplies. In addition, obtaining results can take several hours or even days, making it difficult to provide immediate screening when quick decisions are needed. These limitations highlight the need for alternative approaches that are faster, easier to access, and capable of supporting large-scale public health monitoring.

1.2 Relevance to Artificial Intelligence

To address the above mentioned challenges, this project explores a fully digital approach based on the analysis of cough sounds. Respiratory diseases such as COVID-19 can affect the lungs and airways, causing subtle changes in the way a person coughs. Although these differences are often too small to be noticed by humans, they may contain useful information that can help distinguish healthy individuals from infected patients.

Recognizing these hidden patterns is a task that is naturally tied to artificial intelligence. By converting audio recordings into visual representations, known as Mel-spectrograms, cough sounds can be treated as images and analyzed using computer vision techniques. This allows the system to learn characteristic patterns in the data and use them to support preliminary health screening.

1.3 Project Goals and Student Contributions

The main goal of this project is to build a simple and reproducible screening pipeline that can still perform reasonably well on real-world, noisy audio data. Instead of focusing on creating a completely new model, the contribution of this work lies in adapting an existing approach and improving it so that it works better in specific practical conditions.

- **Model Adaptation for Audio Data:**
  Since the chosen model, ResNet18, was originally designed for standard images, a few adjustments were needed to make it suitable for spectrogram-based inputs. In particular, in this project it was adapted to handle single-channel (grayscale) representations of audio, while still benefiting from the useful visual patterns it had already learned during pretraining.
- **Data Robustness Through Augmentation:**
  To help the system handle limited and noisy data, simple augmentation was introduced, with the techniques applied directly to the spectrograms. These modifications slightly alter parts of the input during training, helping the model become less sensitive to small variations and more focused on meaningful patterns in the sound.
- **End-to-End Usage Pipeline:**
  Finally, the project includes a complete workflow that allows audio to be recorded, processed, and classified in a straightforward way. This makes it possible to test the system on new recordings in a simple and direct manner.

1.4 Report Structure

The rest of this report is organized as follows: Section 2 reviews classical and modern related work in acoustic AI health screening. Section 3 details our methodology, including the signal processing math and the architecture adaptations applied to ResNet-18. Section 4 presents the experimental setup, parameters, quantitative metrics, and a

critical discussion of the results. Finally, Section 5 concludes the report and highlights directions for future development.

2. Related Work

2.1 Classical and Early Approaches

Earlier attempts at detecting illnesses from cough or breathing sounds relied on fairly simple, hand-crafted rules. Instead of learning directly from the audio itself, researchers would first measure a few basic characteristics of the sound, such as how its pitch changes, or how the energy of the signal varies over time. These values were then used as input for traditional machine learning methods to make a final prediction about a possible condition.

While this approach was relatively straightforward, it depended heavily on manually choosing which aspects of the sound were important. In practice, this made it difficult for such systems to deal with the complexity of real cough recordings, which can vary a lot from person to person and from one environment to another. As a result, these early methods often struggled to perform reliably outside of controlled or simplified settings.

2.2 Recent Deep Learning Methods

More recent approaches have shifted away from manually designed features and instead rely on learning patterns directly from the data. A common strategy is to convert audio signals into visual representations, where the sound is represented as a time–frequency image. This makes it possible to apply image-based techniques to audio analysis.

Once in this format, modern image recognition models such as ResNet, VGG, and EfficientNet can be reused for classification tasks through transfer learning. These models are typically trained on large image datasets and are therefore already good at recognizing general visual structures like edges, shapes, and textures. When applied to spectrograms, they are able to pick up subtle visual patterns that correspond to different types of sounds, making them effective for audio-based classification tasks as well.

2.3 Relevant Datasets and Benchmarks

The progress in this area has been strongly supported by the availability of large, publicly shared datasets collected from real users around the world. These datasets make it possible to train and evaluate models in more realistic conditions, rather than relying only on small or controlled recordings. Some of the most commonly used resources include:

- **Coswara**: A crowdsourced dataset collected by the Indian Institute of Science, containing recordings of coughs, breathing, and voice samples from participants worldwide.

- **COUGHVID**: A large open dataset with tens of thousands of cough recordings collected from users in uncontrolled environments.
- **DiCOVA Challenge datasets**: Benchmark datasets designed specifically for comparing different approaches to detecting COVID-19 from respiratory sounds.

These datasets have played an important role in standardizing evaluation and allowing different methods to be compared under similar conditions.

2.4 Limitations of Existing Literature

Despite encouraging results reported in recent studies, there are still several important challenges that limit how well these systems work in real-world conditions. One of the main issues is the lack of large, well-balanced datasets. In most cases, there are far more recordings from healthy individuals than from people who are actually ill. This imbalance can lead models to become biased toward the majority group, making them less reliable when it comes to detecting positive cases.

Another common difficulty comes from the nature of real-world recordings. Unlike controlled environments, crowdsourced audio is often captured in everyday settings where background noise, room acoustics, and differences in recording devices can vary a lot. These inconsistencies make it harder for systems to generalize beyond the data they were trained on, which can significantly affect performance in practice.

2.5 How This Project Differs

This project is designed around the limitations discussed above, with the goal of making the system more reliable on small and realistic datasets. Instead of relying only on raw audio inputs or basic preprocessing steps, we work with spectrogram representations and focus on improving how the model learns from them during training.

A key element of this approach is the use of SpecAugment-style masking, where small random parts of the spectrogram are hidden during training. This simple idea helps the model avoid depending too much on specific details in the data and instead encourages it to learn more general and stable patterns that are less affected by noise or recording conditions. The model itself is based on a ResNet-18 architecture, which provides a strong and stable starting point, especially when working with limited data. In this case, it is not about building a very complex model, but about using a reliable baseline and making it work better in a constrained setting.

3. Methodology

This section describes the full pipeline used to process cough recordings and perform automated COVID-19 screening. The workflow is designed as a sequence of clear steps, starting from raw audio input and ending with a final classification output. In general, the system can be divided into five stages: audio standardization, conversion into a visual representation, data augmentation, feature extraction, and final prediction.

3.1 Overall Pipeline Architecture

The system takes a short cough recording as input and produces a binary output (Healthy or COVID-19 Positive). Each stage of the pipeline is responsible for transforming the data into a more suitable form for analysis.

- **Audio Standardization:**
  Each audio file is first loaded and standardized to a fixed sample rate (22050 Hz) and fixed duration (5 seconds). This ensures that all inputs have a consistent format before further processing.
- **Audio-to-Visual Transformation:**
  The standardized 1D audio signal is converted into a 2D grayscale Mel-spectrogram, which represents sound as a time–frequency image. To make processing more efficient and easier to reuse during training, these spectrograms are saved in a structured numerical format (.npy files). This allows faster access and more stable handling of the data during model training.
- **Data Augmentation (Training Only):**
  During training, random time and frequency masking is applied to the spectrograms. This helps simulate variations in real-world recordings and reduces the risk of the model relying too heavily on specific or irrelevant patterns in the data.
- **Feature Extraction with ResNet-18:**
  The processed spectrogram is then passed through a modified ResNet-18 model adapted to accept single-channel (grayscale) inputs. This component extracts meaningful visual features from the spectrogram representation.
- **Classification Output:**
  Finally, a classification layer uses the extracted features to produce a prediction indicating whether the input cough is classified as Healthy or COVID-19 Positive.

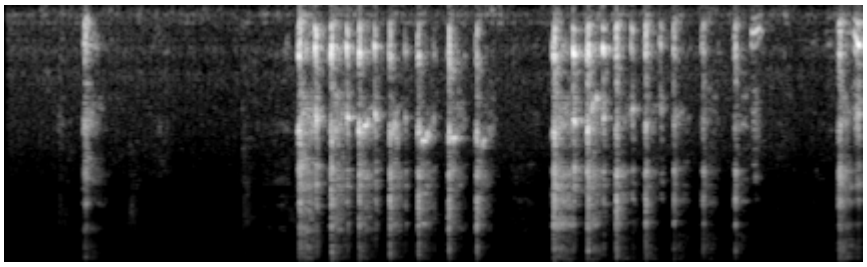
3.2 Input Data and Preprocessing Operations

Since cough recordings collected from real users can vary a lot in length, volume, and quality, the first step is to standardize them into a consistent format before they are processed by the model.

- **Audio Standardization and Resampling:** Each audio file is loaded and resampled to a fixed sampling rate of 22,050 Hz. To ensure consistency across all inputs, we also enforce a fixed time window of 5 seconds. Audio clips shorter than this duration are padded with silence, while longer recordings are trimmed. Earlier

experiments with a 3-second window showed that important parts of the cough cycle were often cut off, which reduced performance. Increasing the window to 5 seconds allowed the system to capture a more complete respiratory pattern, leading to more reliable inputs for the model.

- **Mel-Spectrogram Conversion:** After standardization, the 1D audio signal is converted into a 2D Mel-spectrogram using 128 frequency bands, with a maximum frequency limit of 8,000 Hz. This transformation represents sound in a time–frequency format, making it easier to capture patterns that are not obvious in the raw waveform. The resulting values are converted into a decibel scale, and the spectrogram is then resized into a fixed 128×128 grayscale image to ensure a uniform input shape.



- **Normalization**: To improve numerical stability during training, pixel values of the spectrogram are scaled to a range between 0 and 1 using min-max normalization. A small epsilon value is added in the denominator to avoid division-by-zero errors when the signal has very low variation.
- **Dataset Loading and Labeling:** The dataset is organized into two classes: healthy and COVID-19 positive. During loading, audio files are automatically collected from their respective folders and paired with their labels. Each sample is then passed through the preprocessing pipeline before being used by the model.
- **Data Augmentation (Training Phase Only):** To improve robustness, a simple augmentation strategy is applied during training. This involves randomly masking small sections of the spectrogram in both time and frequency directions. This helps simulate variations in real-world recordings and reduces the model's tendency to rely on small or irrelevant details in the data.

3.3 Structural Augmentation (SpecAugment Pipeline)

Because the dataset is relatively small, a standard model would easily overfit by learning irrelevant details such as background noise or recording conditions instead of meaningful cough patterns. To reduce this risk, we apply a SpecAugment-style augmentation strategy directly to the spectrogram representations during training. This approach introduces controlled randomness by modifying the input data in two main ways:

Frequency Masking: Random frequency ranges (up to 20 bins) are temporarily masked in the spectrogram. This effectively hides certain pitch-related components of the sound, encouraging the model to rely on broader spectral patterns rather than specific frequency details.

Time Masking: Random time segments (up to 20 frames) are also masked. This removes short portions of the cough signal, helping the model become less sensitive to exact timing and more focused on overall structure.

Together, these transformations make the model more robust to variation in real-world recordings. By preventing it from depending too heavily on small or specific patterns, the system is encouraged to learn more general and stable representations of the data.

3.4 Model Architecture & Transfer Learning Adaptation

The core component of this system is based on the ResNet-18 architecture, which is commonly used for image classification tasks. However, since ResNet is originally designed for standard RGB images, some adjustments were necessary to make it suitable for spectrogram-based inputs.

- **Adapting the Input Layer:** Standard ResNet models expect three-channel (RGB) images, while our spectrograms are single-channel grayscale representations. To handle this, the first convolutional layer was modified to accept one input channel instead of three, while keeping the same kernel size and structure. This allows the model to process spectrograms without changing the overall architecture.
- **Using Pretrained Knowledge:** Instead of initializing the modified layer from scratch, we reused the knowledge learned from pretraining on large image datasets. The original RGB filters were averaged into a single channel so that the model could retain its ability to detect basic visual patterns such as edges and textures, even when working with grayscale inputs.
- **Classification Layer Adjustment:** The final part of the network was replaced to match the task at hand. The original classification layer was removed and substituted with a simpler head designed for binary prediction. This includes a dropout step to reduce overfitting, followed by a final linear layer that outputs two possible classes: Healthy or COVID-19 Positive.

Overall, these changes allow a standard image-based model to work effectively on audio-derived spectrograms while keeping the benefits of pretrained visual feature extraction.

3.5 Objective Function and Training Details

- **Optimization Setup:** The model is trained using the Adam optimizer, which is a commonly used optimization method in deep learning because it automatically adapts the learning rate for each parameter during training. This makes it particularly useful in situations like ours, where the dataset is relatively small and noisy, since it helps the model converge more smoothly and reduces the risk of unstable updates. We use an initial learning rate of 0.0005 and a weight decay value of 1e-4 to further control overfitting. Training is carried out over a maximum of 20 epochs with a batch size of 8.

- **Adaptive Learning Rate and Model Saving:** To improve training stability, a learning rate scheduling strategy is used that monitors validation performance during training. When progress slows down, the learning rate is automatically reduced to allow the model to make finer adjustments. In addition, the model is saved whenever it achieves a new best validation performance, ensuring that the most effective version is preserved for later use.

Overall, these training choices are designed to make the model more stable and reliable, especially given the limited and imbalanced nature of the dataset.

4. Experimental Setup & Results

This section describes the testing framework used in our experiments, including how the dataset was prepared, the configuration settings, and how the proposed pipeline performs under controlled evaluation conditions.

4.1 Dataset

For our experiments, we used the public "COVID-19 Cough Sounds" dataset available on Kaggle. This dataset aggregates crowd-sourced recordings from sources such as Coswara and Virufy, making it a realistic but naturally noisy and inconsistent collection of respiratory audio samples.

Since the raw data is highly unbalanced and not fully clean, a preprocessing step was applied to make it suitable for evaluation. First, all recordings with unclear or missing labels (such as those marked "unknown") were removed to ensure that only reliable samples were included. After this filtering step, the dataset still showed a strong class imbalance, with a large majority of healthy recordings compared to positive COVID-19 cases.

To avoid biasing the model toward the majority class, we applied random downsampling of the healthy group. This resulted in a balanced dataset containing 150 healthy recordings and 150 COVID-19 positive recordings. The final dataset therefore consists of 300 audio samples in total, stored in common formats such as .wav, .mp3, and .m4a, recorded using a variety of devices and environments.

For evaluation, we used a standard 80/20 stratified split, ensuring that both classes were equally represented in the training and validation sets. This resulted in 240 training samples and 60 validation samples, with no overlap between the two sets.

Table 1: Dataset Summary

Property	Value
Dataset name and source	COVID-19 Cough Sounds (Kaggle / Pranayanandan63)
Number of samples	300
Number of classes	2 (Healthy / COVID-19 Positive)
Training samples	240 (80%)
Validation samples	60 (20%)
Input format	Single-channel Grayscale Mel-Spectrogram (128x128)

4.2 Training and Evaluation Protocol

All experiments were carried out locally on a standard 64-bit Windows laptop, without the use of dedicated GPU acceleration. This setup was intentionally chosen to reflect a lightweight and realistic scenario, similar to how a simple diagnostic tool would run on everyday consumer hardware.

The training configuration was kept consistent across all runs. The model was optimized using the Adam optimizer with an initial learning rate of 0.0005 and a small weight decay of 1e-4 to help reduce overfitting. Training was performed using mini-batches of 8 samples over a maximum of 20 epochs. To further improve stability during training, a ReduceLROnPlateau scheduler was used to monitor validation performance and reduce the learning rate when progress stalled.

To evaluate the effectiveness of the proposed pipeline, we also compared it against a simpler baseline model trained under the same conditions. This baseline consists of a basic convolutional neural network with four layers,

initialized from scratch without any pretraining or augmentation strategies. Unlike our proposed approach, it does not use SpecAugment-style masking or transfer learning, making it a useful reference point for measuring the impact of our design choices.

4.3 Results

The quantitative results show that using a pre-trained computer vision model provides a clear improvement over a standard network trained from scratch.

The vanilla CNN baseline quickly began to overfit on the small dataset, reaching a plateau in performance at around 70.0% validation accuracy. In contrast, our modified single-channel ResNet-18 approach achieved a peak validation accuracy of 86.7%, demonstrating a stronger ability to generalize and distinguish between the two classes.

Table 2: Comparison Between Baseline and Proposed Approach

Vanilla CNN Baseline: 70.0% Accuracy | 0.71 Precision | 0.69 Recall | 0.69 Macro F1-Score

```
              precision    recall  f1-score   support

     Healthy       0.75      0.71      0.73        34
COVID-19 Positive       0.65      0.69      0.67        26

    accuracy                           0.70        60
   macro avg       0.70      0.70      0.70        60
weighted avg       0.71      0.70      0.70        60

Confusion matrix for best epoch:
[[24 10]
 [ 8 18]]
```

Proposed Method: 86.7% Accuracy | 0.86 Precision | 0.86 Recall | 0.86 Macro F1-Score

```
              precision    recall  f1-score   support

     Healthy       0.88      0.88      0.88        34
COVID-19 Positive       0.85      0.85      0.85        26

    accuracy                           0.87        60
   macro avg       0.86      0.86      0.86        60
weighted avg       0.87      0.87      0.87        60

Confusion matrix for best epoch:
[[30  4]
 [ 4 22]]
```

Looking more closely at class-level performance at the best epoch (epoch 20), the results show that the class balancing strategy helped maintain consistent performance across both categories:

Healthy Class: 0.88 Precision | 0.88 Recall | 0.88 F1-Score (34 samples)
COVID-19 Positive Class: 0.85 Precision | 0.85 Recall | 0.85 F1-Score (26 samples)

$$M = \begin{pmatrix} 30 & 4 \\ 4 & 22 \end{pmatrix}$$

From the confusion matrix results on the 60 validation samples, the model correctly classified 30 healthy cases and 22 COVID-19 positive cases. The remaining predictions consisted of 4 false positives and 4 false negatives, showing a fairly balanced error distribution across both classes.

```python
#3-layer CNN model
class SpectrogramCNN(nn.Module):
    def __init__(self):
        super(SpectrogramCNN, self).__init__()

        #Conv layer 1
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        #After two Max pooling layers
        self.fc1 = nn.Linear(32 * 32 * 32, 128)
        self.fc2 = nn.Linear(128, 2)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))

        #Fully connected layer
        x = x.view(-1, 32 * 32 * 32)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
```

4.4 Critical Discussion

The results obtained in this study suggest that adapting computer vision models for audio classification can be an effective approach for medical screening tasks, particularly when working with relatively small datasets. The observed improvement of 16.7 percentage points over the baseline CNN indicates that transfer learning provides a strong starting point when only a limited number of training samples are available.

One possible explanation for this improvement is that the pre-trained ResNet-18 model already contains useful low-level feature detectors, allowing it to capture meaningful patterns in the spectrograms more efficiently. In addition, the use of SpecAugment-style time and frequency masking helps reduce overfitting by encouraging the model to focus on more general characteristics of the cough signal rather than on specific recording conditions.

Error Analysis and Observed Limitations: An examination of the eight misclassified validation samples revealed several challenges commonly encountered in crowdsourced respiratory datasets:

Acoustic Noise Interference: Real-world recordings often contain background speech, room echoes, or environmental sounds. These artifacts may overlap with the cough signal in the spectrogram and make it more difficult for the model to identify relevant features.

Recording Device and Volume Variability: Since the dataset contains recordings captured on different devices and stored in multiple formats, there are noticeable differences in recording quality and signal intensity. Even after normalization, very quiet or compressed recordings may contain less distinctive information.

Fixed-Length Window Effects: Using a fixed 5-second input window ensures consistent dimensions across all samples, but very short cough recordings may contain large regions of silence after padding. As a result, the amount of useful information available to the model can vary between recordings, occasionally leading to incorrect predictions.

Despite these challenges, the relatively balanced class-specific F1-scores (0.88 and 0.85) indicate that the proposed pipeline maintains consistent performance across both classes and does not exhibit a strong bias toward either category.

5. Conclusion

This project demonstrates that computer vision techniques can be successfully applied to respiratory audio analysis for preliminary COVID-19 screening. By converting one-dimensional cough recordings into two-dimensional Mel-spectrogram representations, we were able to adapt a pre-trained ResNet-18 model to work with single-channel audio-derived images. To improve robustness on a limited dataset, we combined a balanced training set with SpecAugment-based masking techniques during training.

Although this work represents a proof of concept rather than a clinical diagnostic tool, the results indicate that non-invasive and easily accessible screening methods based on cough recordings have considerable potential. More broadly, the project highlights how existing computer vision architectures can be adapted to address problems beyond traditional image analysis.

5.1 Future Work

While the proposed system produced encouraging results, several aspects could be improved in future work.

1. **Multi-Disease Classification:**
   The current model is limited to distinguishing between healthy individuals and COVID-19 positive cases. With access to larger and more diverse datasets, the framework could be extended to recognize additional respiratory conditions such as asthma, bronchitis, or pertussis.

2. **Improved Noise Handling:**
   Since real-world recordings are often affected by background sounds and differences in recording quality, incorporating dedicated noise reduction techniques during preprocessing could further improve robustness.

3. **Lightweight Architectures for Mobile Deployment:**
   Future work could also investigate smaller architectures, such as MobileNet, to reduce computational requirements. This would make it easier to deploy the system on resource-constrained devices and support fast, on-device screening without relying on external servers.

References

1. *[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.*
2. *[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.*
3. *[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.*
4. *[4] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in European Conference on Computer Vision (ECCV), pp. 740–755, 2014.*
5. *[5] P. Nandan, "COVID-19 Cough Sounds Dataset," Kaggle Repository, 2024. Available: https://www.kaggle.com/datasets/pranaynandan63/covid-19-cough-sounds.*